# Information Technology Social Systems
# The Mycite Project Mission & A Disambiguous Universal Language of Information

Dylan Montgomery

**Abstract**

We present the Mycite Framework, a universal language of information (ULI) based on an information-theoretic schema for sequential bitstreams. The framework employs the MSS convention, which encodes hierarchy, structure, and semantic boundaries directly within the data stream, enabling retrodeterministic interpretation and polymorphic recombination without fixed word-size assumptions. By unifying data and metadata into a single representation, the system supports semantic interoperability across heterogeneous hardware and software platforms, reducing reliance on translation layers and rigid standards. This design allows software and interfaces to adapt dynamically to context, improving accessibility and modularity. The framework also provides a semantic substrate for distributed systems in which nodes coordinate meaning through shared grammar rather than global consensus, offering an alternative to centralized or blockchain-based architectures. Applications include digital libraries, AI reasoning, and cooperative information networks, with broader implications for decentralized information economies.

**Keywords:** universal language of information, MSS convention, bitstream notation, semantic networks, interoperability, information theory, digital systems

## 1 The MSS Convention

Digital computers, in combination with laws of information, provide a stringent guide that one must stay within to create a ULI. The MSS is a method of implementing a self organizing universal language of information.

**Parameters.**

- **Self-terminating**
- **Retrodeterministic**
  The meaning of any part of a sentence cannot be understood as intended until the sentence signals that it is finished; either by pause or by contextual conclusion. This is the result of not using a word size convention, so that the convention is not limited by any size parameter.
- **Polymorphic Combinitoric Continuity**
  In tandem with no size constraints or use of word size, we are able to employ a system of recombination. This means that every layer of any of the possible instances of an abstraction's notation can be used to define a new instance in a later abstraction. (Only possible for application with a clean up carry over bit mask.) Progressively zoomable: higher layers refine information without rewriting earlier ones.

With these as a base, one can describe a ULI that takes the form of a sequential stream of 1s and 0s, one that can then notate any instance of information. However, there are many notable conventions used for the particular ULI that is discussed later. This is to avoid the explosive size of notating ever more complex abstractions of rearrangement.

## 1.1 Sequential Bit Stream Interpreting

Let us consider the method in the context upon binary file acquisition. Firstly, beginning by reading the contents of a binary file and expressing the data as a flat sequence of bits. This transforms raw byte-oriented storage into a bit-level stream suitable for structural interpretation. With these assumptions in place we can apply a methodology to interface with the Mycelium Schema Standardization.

### 1.1.1 Initial Segmentation

- **Detection of Index A:** From the beginning of the bitstream, identify the run of consecutive 1s until the first 0 is encountered. This initial run constitutes Index A. The length of this run (the count of 1s) is significant: it defines a constant referred to as the address size. Going forward, all structural addresses in the stream will be expressed in blocks of this length.
- **Extraction of Index B:** Immediately following Index A and its terminating 0, the next block of bits of length equal to the address size is isolated. Interpreted as a binary integer, this block yields Index B. The semantic role of Index B is to denote the file size or overall length of the bitstream.
- **Construction of Index C:** After Index B, the next two consecutive blocks, each of length equal to the address size, are extracted. Together, these four binary integers form Index C. Each element of Index C specifies a non-inclusive stopping position for one of the subsequent major sections of the stream.
- **Application of Index C:** The four values in Index C are then interpreted sequentially as delimiters for the following indexes: the first value specifies the endpoint of Index D; the second value specifies the endpoint of Index G. Both of these indexes is defined as the segment of the stream between successive stopping positions, with boundaries determined by the corresponding values in Index C.

**Key Observations.**

- *Run-Length Encoding of Address Size:* The number of leading 1s is not just a marker but a rule-setting mechanism. It dictates the granularity of subsequent address-based parsing.
- *Self Organizing Structure:* The stream encodes both its own size and the boundaries of its subsections within its first few blocks. This allows any compliant interpreter to reconstruct the structure without external metadata.
- *Hierarchical Partitioning:* Detect Index A (to establish address size). Read Index B (file length). Read Index C (subsection boundaries). Use Index C to partition the remaining stream into distinct indexes (D, G, E, S).

### 1.1.2 Further Interpreting

Begin by interpreting an initial portion of the sequential bitstream as a control value. This control value specifies how many high-level structures (for example, layers) will exist in the nested representation.

- **Layer Definition:** For each declared layer, the next portion of the bitstream encodes how many substructures (for example, groups) that layer contains. Each group count is drawn sequentially from the stream, so the stream itself dictates the topology.

- **Group Definition:** For each group, another segment of the stream specifies how many iterations (repeated sub-units) it will hold. Again, the count of iterations is not arbitrary; it is derived directly from successive values in the stream.
- **Iteration Population:** Once the structure of layers and groups is defined, subsequent segments of the bitstream are consumed in fixed-size units (in this example, pairs of values). Each unit corresponds to the contents of an iteration. Thus, the raw stream is gradually partitioned: first to set the hierarchy, then to fill it with data.
- **Hierarchical Assembly:** The nested representation is constructed by associating each extracted value with its ordinal position (for example, layer 1, group 2, iteration 5). This ordinal indexing provides unique keys for every element in the hierarchy, regardless of the actual bit values. The result is a tree-like structure in which the bitstream is both the blueprint (defining counts) and the content (providing payload values).

**Key Principle.** The process alternates between two modes: counting mode (determine how many children or iterations to expect at the next level) and filling mode (consume the specified number of payloads). Because the stream is sequential, each mode consumes exactly as much data as is needed before moving forward.

A good rule of restraint is in trying to apply a system that allows for the extrapolation of building blocks. This allows for the mapping of values without recombination. Then use the post genesis name space to recombine for data storage and use. Since, written language is attributed

Initial value notations within the schema must extend themselves to maximize their notation, with respect to space, by aligning with digital limiters. The value notations must maintain retroindeterministic behavior. To do this we use unitary prefacing. Urinary prefacing must expand the magnitude in which a following denotation can represent at ever increasing rates of continued prefacing. This can only be done by representing an increasingly increasing constant rate of notation that sets the frame for the following notation. (This is not an increasingly increasing constant rate of value, but rather the manner in which each respective sequential 1, or TRUE bit, can further instill in what is to be expected. All of which is up until a 0, or a FALSE bit, is encountered.)

Therefore, the balance in this growth must consider: What are the informational limits of representing a continuous natural number field array?

To isolate this limiter, we can consider each cell of notation as an incremental addition to the informational capacity of a notation. { See Markov Chains and or Stochastic Processes} Now if we use a convention that employs a capacity to add or subtract by 1, then we can create paths to values within an instilled field that more efficiently arrives upon a greater number of values.

## 1.2 Data Semantic Networks

Instead of each piece of hardware or software defining its own format and requiring custom translation layers, we use a shared data standard, so every system can speak the same language natively. However, even with data standardization, each standard still limits what can be expressed and translation between standards is not always possible if they capture different assumptions or dimensions of the data. Hardware and software each follow their own internal logic and data handling standards, using external standards only at the point of transmission. So, the meaning of data is local to the system's operational context, not to the standard it is packaged in.

Currently, hardware and software systems cannot achieve indiscriminate interoperability by default and they must be explicitly engineered to interact in specific ways. This is because there exists no universal data standard capable of expressing the infinite variety of possible data

structures, contextual relationships, operational states, and localized identifiers that define how systems behave.

Internally, every system handles data in a way that is deeply tied to its own logic and operational flow. External data standards, when used, merely serve as fixed formats for transmission, not for internal interpretation or function. As a result, systems cannot natively interpret or manipulate arbitrary data or behavior without prior integration work.

This has real-world consequences. For instance, software interfaces (especially graphical user interfaces or GUIs) are built with hard-coded assumptions about purpose, data types, and interaction models. Because the GUI is not modularized by function or semantic context, it cannot automatically reconfigure itself to support different modes of interaction, such as non-visual navigation. This is a key reason digital systems remain inaccessible to many visually impaired users.

In short, unless explicitly designed to expose their structure, behavior, and data semantics in a modular and machine-readable way, software systems cannot adapt their interface or behavior to new contexts. The limitation is not just technical, it is foundational to how we currently design systems: for specific uses, in specific forms.

### 1.2.1 The Technical Core

The vision of the semantic web has long been to create a more intelligent and intuitive internet by structuring data in a way that machines can understand context and relationships. The concept is meant to create a web where data is structured and tagged with metadata in a way that encodes relationships for the meaning, and context between different pieces of information. However, it has faced many challenges in terms of implementation. The foremost hurdle being the complexity of structuring all web data in a universally understood format, and the sheer scale of the web makes it challenging to adopt such a system universally. This leads many to believe it is not a viable approach on a large scale. Instead, more focused applications, like structured data for search engines, have become the practical route forward.

In contrast, the Mycite Framework demonstrates that a semantic data structure is indeed possible and practical. By integrating data and metadata into a single, unified system, the framework uses a highly reduced alphabet of elements to embody the components that are arranged to create any instance of information. This is accomplished by using the SSID Layering to create hierarchical abstractions. This approach uses partial-convention, rather than being solely self defining. This allows for the unrestrictive nature of what can be defined, while also preventing notations from having to notate with impossible specificity that differentiates an instance from infinity.

### 1.2.2 MSS Interface Development

The significance of being able to reduce all information down into a universal language is that it is made up of a finite set of components that when reneged, can make up any instance of information. In this way, all encodings of information can be represented as locations that are navigated to and all creations are discovered. This allows for encoding of information in memory as mapped out location within this space. In doing so it can be made immediately parents how adjacent locations are contextualized with each.

This is also the same reasoning with which the MSS software is made to handle and operate on data without being explicitly engineered to. This makes the task of software development far faster for any given purpose and context. Rather than having to consider how to visualize data, navigate it, and mediate on it for each case; the development process foremost includes considerations for how a user may want to more complexly interface with data, rather than what form it expects information to exist in.

In this way, whether software is made to operate as a computer directed process or if it involves a human interface, the software framework can stay the same for almost any case. This is what is called the MSS Control Gate Framework (GCF). The GCF functions as an input abstraction layer; where no matter if it is mouse, keyboard, or an API, it comes through as a unified intention type, like navigation, investigation, mediation or manipulation. In this way, the GCF can have software developed over top of it to handle that intention in context, without prior context. This is all possible as the MSS is accessible as if it were a single instance of data, where interfacing allows for the GCF to handle all data and its structures as an ordered instance is abstracted from the observable components it is made from. This means the application state, like the current location of investigation (that is, the directory) or aspects of child objects that may be selected, they all shape how the system responds to output to a display or terminal.

Additionally this can also be used to deepen the understanding an AI's Neural network has about a singular topic and its relation to other ideas.

> Some people can read *War and Peace* and come away thinking it is a simple adventure story... Others can read the ingredients on a chewing gum wrapper and unlock the secrets of the universe.
> *Lex Luthor in the 1978 movie Superman*

# 2 Agricultural Market Microstructure

We propose a transparent, data-driven agricultural exchange; an *agricultural market microstructure*; inspired by the mechanics of modern securities markets. Every producer's output is visible, comparable, and dynamically valued in context (geography, timing, logistics, and sustainability attributes). The system aims to couple the allocative efficiency of electronic markets with the ecological and social resilience of local economies.[1]

## 2.1 Agriculture as an Information Network

Stock exchanges evolved by standardizing information and continuously matching orders so that prices reflect dispersed knowledge. In the same spirit, a *Mycelial Exchange Model* treats agriculture as an information network:

- **Nodes:** Each farm or cooperative is a node listing quantified, geotagged supply units (bushels, pounds, crates) with rich metadata (location, production method, certifications, sustainability practices).
- **Continuous signals:** Consumers, retailers, and distributors generate continuous demand signals; producers emit continuous supply/availability signals.
- **Matching engine:** A continuous auction matches these flows; data largely replaces manual intermediaries; so local exchanges clear first when efficient.[2]

In financial markets, trade occurs because beliefs and constraints differ, and execution is optimized *over time*, not all at once. Smooth, continuous matching reduces frictions and lets the system discover context-specific value in real time; paralleling equilibria in which inventories adjust gradually while prices immediately reflect aggregated information.[3]

[1] Albert S. Kyle, *Continuous Auctions and Insider Trading* (Econometrica, 1985).
[2] Albert S. Kyle, *Informed Speculation with Imperfect Competition* (The Review of Economic Studies, 1989).
[3] Albert S. Kyle and Anna A. Obizhaeva, *Smooth Trading with Overconfidence and Market Power* (The Review of Economic Studies, 2018).

### 2.1.1 A Parallel to the Evolution of Stock Pricing

- **17th–19th c.:** Joint-stock forms and tickers standardized ownership and prices.
  *Agricultural parallel:* A unified data fabric standardizes descriptions of yield, surplus, and geographically resolved demand in near real time.

- **20th c.:** Clearing and order tracking ensured integrity and accountability.
  *Agricultural parallel:* Independent producers can market under shared *quality-standard brands* (co-op labels with enforceable practices), preserving authenticity and reducing waste/fraud in pooled offerings.

- **21st c.:** Electronic markets deliver transparent price discovery and liquidity.
  *Agricultural parallel:* A digital food-exchange layer dynamically adjusts prices to local conditions (distance, perishability, time windows), matching nearby demand first and routing surplus to processors, food banks, or secondary markets.[4]

In both systems, *information friction*; ignorance of what exists where and when; is the main inefficiency. A continuous, standardized data layer removes that friction and makes trade feasible even when participants "agree to disagree" about value; precisely what sustains trading volume in microstructure models.[5]

## 2.2 Economic Properties of the Paradigm

**Transparency.** Participants see real-time supply and demand with geographic context. Signals become local and time-specific rather than coarse national averages.

**Localization & sustainability.** Transportation costs and perishability are *priced in*, so nearer goods typically clear first. In microstructure terms, prices should reflect both *levels* (where inventory sits) and *rates* (how quickly it must move).[6]

**Waste reduction.** Ex-ante visibility of local demand curves curbs overproduction and redirects surplus in time (processors, charities, secondary outlets).

**Preference-aligned selection.** Distinct attributes (organic, regenerative, non-GMO, fair trade) become state variables in the matching function, allowing products to earn premia when buyers value them; fitness is revealed by actual, traceable choices.

**Data layer (market feeds).** A standardized schema (type, quantity, quality, location, timing, certifications) continuously updates via IoT, inventory apps, and co-op registries; akin to feeds that drive continuous auctions and immediate price incorporation in finance.[7]

**Matching & routing.** A matching engine (analogous to a limit-order book) pairs supply and demand subject to distance, time windows, perishability, and capacity constraints, jointly solving for price and route. Execution speed is costly: pushing large volumes *too fast* relative to local absorption capacity depresses effective prices temporarily, while steadier routing realizes better average outcomes; mirroring the distinction between *permanent* and *temporary* price impact.[8]

The modern information economy is dominated by centralized platforms that contain rather than coordinate. Across sectors, these systems impose artificial drag: duplicated efforts, incompatible standards, and opaque supply chains. In such architectures, waste is not incidental; it is structural.

In agriculture, despite strong demand for low-chemical inputs, heirloom varieties, and sustainably grown produce; and clear community value from local economies; small farms

[4] Albert S. Kyle and Anna A. Obizhaeva, *Market Microstructure Invariance: Empirical Hypotheses* (Econometrica, 2016).

[5] Kyle (1989). See also Kyle–Obizhaeva (2018) on intertemporal execution and belief heterogeneity.

[6] Kyle–Obizhaeva (2016, 2018).

[7] Kyle (1985); Kyle–Obizhaeva (2016).

[8] Kyle–Obizhaeva (2018).

frequently achieve high output per acre in perishables yet face coordination barriers. Without visibility into regional demand and flexible tools for inter-farm routing and aggregation, local supply fails to meet local demand. Communication remains fractured; software ecosystems are rigid and often optimized for large, centralized actors. Small producers cannot coordinate at scale, and large producers cannot adapt without imposing full control.

The observed "productivity gap" is thus less a technological limit than an accessibility problem: small agriculture is obstructed from being collectively discoverable and schedulable. That discourages utilization (perishables spoil without known buyers) and constrains planting decisions (acreage left underutilized due to uncertain offtake).

Where coordination is most needed, rigidity compounds failure. A semantic internet provides a structural alternative: instead of forcing participants to conform to a single platform, it makes meaning legible across systems. Semantics becomes the bridge between diverse actors, eliminating the need for uniform infrastructure.

History rhymes. Centralization initially delivers scale economies (as with early automotive mass production) but later becomes a bottleneck to innovation. Today, individuals can produce studio-grade media with tools that once required entire teams, yet administrative infrastructure is re-centralizing under monopolistic control. The pattern repeats: efficiency first, then stagnation.

Globalized supply chains remain locked into scale-driven visibility. Multinationals outcompete not merely by product, but by top-down access to demand and supply information. If economies had instant, shared transparency, competition would shift toward quality and contextual fit. Today, only corporate silos achieve this coordination.

A semantic architecture changes this. Shared grammar enables cooperation without centralized control. Local businesses retain autonomy while achieving scale-like coordination. Waste declines, adaptability rises, and global reach is no longer exclusive to monopolies. In short, shared grammar enables cooperation without monolithic platforms; centralized architectures create drag and spoilage by design.

### 2.2.1 Valuation Layer

Instead of one market price, value is a contextual function of:

- *Distance and logistics* (embedded transport and perishability costs),
- *Demand intensity* (time-varying local willingness to pay),
- *Attributes* (organic/regenerative/fair-trade certifications).

This mirrors financial microstructure where the *price level* is sensitive to inventory and beliefs, but *execution costs* depend on *how fast* trades occur. In continuous-time equilibria, price impact decomposes into a *permanent* component (linked to inventory levels) and a *temporary* component (linked to execution speed).[9] The agricultural analogue replaces "who can ship the cheapest tomato the farthest" with "who can produce the most contextually valuable tomato for the ecosystem it serves," while optimizing the *speed* and *path* of routing to preserve value.

### 2.2.2 Barcelona Proof-Point

Barcelona's move toward a "produce locally, share globally" model; developed through the Fab City initiative by the Institute for Advanced Architecture of Catalonia (IAAC) and the global Fab Lab network; illustrates how civic-scale economies can decentralize without losing interoperability. It embodies what Neil Gershenfeld describes as a future where data travels globally while materials stay local.[10]

[9] Kyle–Obizhaeva (2016, 2018).

[10] Neil Gershenfeld, Alan Gershenfeld, and Joel Cutcher-Gershenfeld, *Designing Reality: How to Survive and Thrive in the Third Digital Revolution* (Basic Books, 2017).

Rather than rely on centralized production and logistics, Barcelona invests in distributed infrastructure: public maker spaces, open-source toolchains, local fabrication labs, and education. Design files are shared across continents; physical goods are produced in the neighborhood. The city flips the supply chain from importing finished goods to importing instructions. This is more than a policy tweak; it shows that complex, urban-scale economies can decentralize while remaining interoperable, precisely the kind of architecture a semantic internet enables. When meaning is standardized and interaction is legible, coordination no longer requires hierarchy.

# 3 The Mycite Framework

**Thesis.** Whereas blockchain protocols replicate trust through rigid consensus and ledger duplication, the *Mycite Network* achieves distributed cooperation through *semantic alignment*. It is not a chain of blocks; it is a lattice of meaning. Nodes operate independently and interoperate via *semantic diff* rather than absolute uniformity. Each node maintains its own schema and interpretive context. Synchronization emerges not by enforcing a single truth, but by resolving differences through shared grammar. The outcome is *Autonomy*, *Interoperability*, and *Auditability*: trust as an *outcome of interpretability*, not brute replication.

Practically, two regional food cooperatives can share forecasts, inventories, and logistics even if their IT stacks are unrelated. They do not need to run the same software—only to publish and subscribe to a *semantic data fabric*. Governance and privacy remain local; coordination becomes routine.

The Mycite Network is the infrastructure backbone that renders a "Semantic Internet" practical: a living, federated system of meaning rather than a monolith.

## 3.1 Design Goals

- **Autonomy:** Each participant retains its own data models, policies, and operations.
- **Interoperability:** Cross-system exchange is achieved through schema mapping and semantic diffs, not format lock-in.
- **Auditability:** Provenance, versioning, and policy traces are first-class so data is explainable ex post.
- **Minimal Coupling:** No global ledger or global clock; only shared meaning and verifiable interfaces.

## 3.2 Architecture (Layered)

**L0: Identity & Access.** Decentralized identifiers for organizations and services; signed claims; least-privilege tokens for API access.

**L1: Schema & Grammar.** The *Mycelium Schema Standardization (MSS)* defines canonical *vocabularies* (entities, attributes, relations) and *contexts* (geography, time, certification). MSS does not freeze local models; it provides a shared *grammar* to describe them.

**L2: Mapping & Diff.** Bidirectional schema mappings (local $\leftrightarrow$ MSS) and *semantic diff* to reconcile records without forcing identical structures. Conflicts are resolved by policy (priority, recency, authority) with machine-readable justifications.

**L3: Event Bus & Feeds.** Append-only *semantic feeds* (inventory updates, forecasts, orders, routes) with content-addressed payloads. Nodes subscribe to topics (crop, region, certification) and receive normalized deltas.

**L4: Policies & Contracts.** Human-legible, machine-executable rules for data sharing, retention, attribution, and certification checks (e.g., organic/non-GMO). Policies travel with the data as *sticky metadata*.

**L5: Audit & Provenance.** Hash-chained envelopes for versions and signatures (who/what/when/why). No global consensus required; verifiers can recompute integrity off-line and trace transformations end-to-end.

### 3.3 Protocol Primitives

- **Describe(entity)**: Publish local schema fragments with MSS bindings (types, units, geotags, time).
- **Map(local $\leftrightarrow$ MSS)**: Maintain reversible mappings; expose them as versioned APIs.
- **Diff(a,b)**: Compute semantic delta between records; attach policy-guided resolution notes.
- **Prove(record)**: Emit signed provenance (source, method, certification check, transform history).
- **Subscribe(topic, filter)**: Receive normalized events (e.g., "leafy greens, 50mi radius, organic").

### 3.4 From Individual to Network

**Local First, Network Later.** Fruitful Network Development (FND) delivers interactive data tools that *stand alone* yet natively speak MSS. Producers gain value immediately (planning, inventory, routing); when connected, they gain network effects without surrendering control.

**Interfaces, Not Lock-in.** Data remains where it is. Publishing requires only an API or a public webpage with extractable structure; a buyer-facing portal aggregates *on demand*, acting more like a domain-specific search engine than a platform owner.

**Trust in Practice.** Product quality and method are governed via cooperative seals and certification pipelines (analogous to kosher/organic/non-GMO), with criteria encoded as portable policies and verifiable checks.

**Network Effects.** As with social platforms, marginal value grows with participants. Unlike platforms, Mycite preserves *outside-option value*: the tools are useful even in isolation, so adoption can begin without waiting for the network.

### 3.5 Adoption Value: Lean, Plan-Driven Operations

The "workware" provides immediate, off-network utility, modeled after lean farm practice [**?**] and crop-planning best-practice [**?**]:

**Financial Planning & Profitability.** Set targets; itemize costs; estimate gross sales; track seasonally; review profit drivers; iterate annually.

**Market-Back Planning.** Define product/price sheets; choose channels (CSA, market, wholesale); project sales; align plantings to demand; post-mortem each cycle.

**Production Scheduling.** Timelines/calendars for seeding, transplanting, harvest; load leveling (*heijunka*); visual pull systems (*kanban*) to smooth labor and avoid bottlenecks.

**Standard Work & Resource Use.** Consistent bed prep; compost systems with minimal motion/waste; efficient field/greenhouse layouts; checklists and one-point lessons.

**Rotation, Mapping, & Trials.** Block rotations; yearly maps; variety trials with structured notes for next-season selection.

**Greenhouse & Seed Orders.** Forward schedules; quantity/type formats aligned to the plan; pre-validated orders against targets.

**Operational Excellence (Lean 5S).** Sort, set in order, shine, standardize, sustain; remove muda (overproduction, waiting, motion, excess inventory, defects, underused talent).

**Records & Improvement.** In-season capture of plantings/harvests/sales/issues; periodic variance checks; end-of-season reviews feeding next-year plans.

### 3.6 Why Mycite (vs. Consensus Ledgers)

- **Flexibility:** Heterogeneous schemas evolve without forks; mappings absorb change.
- **Scalability:** No universal consensus loop; only parties to an exchange validate what they need.
- **Explainability:** Semantic diffs and provenance make disagreements legible and resolvable.
- **Privacy by Design:** Policies bind to data; selective disclosure replaces global replication.

### 3.7 Governance & Assurance

Cooperative standards are codified as machine-checkable policies (criteria, evidence, review cadence). Issuance/revocation of seals is recorded as signed claims; buyers can verify claims without revealing private data. This yields credible signaling without centralized custody.

### 3.8 Implementation Sketch

1. Publish MSS v1 vocabularies (entities, units, geo/time, certifications) and mapping templates.
2. Ship FND "workware" modules (planning, inventory, routing) with MSS-native export.
3. Stand up a semantic event bus (topic registry, filters, signatures, provenance).
4. Onboard early co-ops; encode their standards as policies and test seal workflows.
5. Iterate on mappings and diff policies; add buyer portal for query-time aggregation.

## 4 License

Licensed under the Apache License 2.0: https://www.apache.org/licenses/LICENSE-2.0. This allows modification, redistribution, and commercial use with attribution.

## 5 Acknowledgments

Built and authored by Dylan Montgomery.
Version: 9.03.04 Status: Active prototyping and architectural refinement.

## References